

Faculdade de Engenharia da Universidade do Porto



Modelação de uma Base de Dados para o Jogo *Clash Royale*

FEUP-L.EIC012-2025/2026-1S - Bases de Dados
Licenciatura em Engenharia Informática e Computação

Grupo 2LEIC012_0802

Supervisor

Lázaro Gabriel Barros da Costa

Estudantes e Autores

Francisco Campos Lourenço : up202404899@up.pt

João Pedro Lopes Martins : up202403610@up.pt

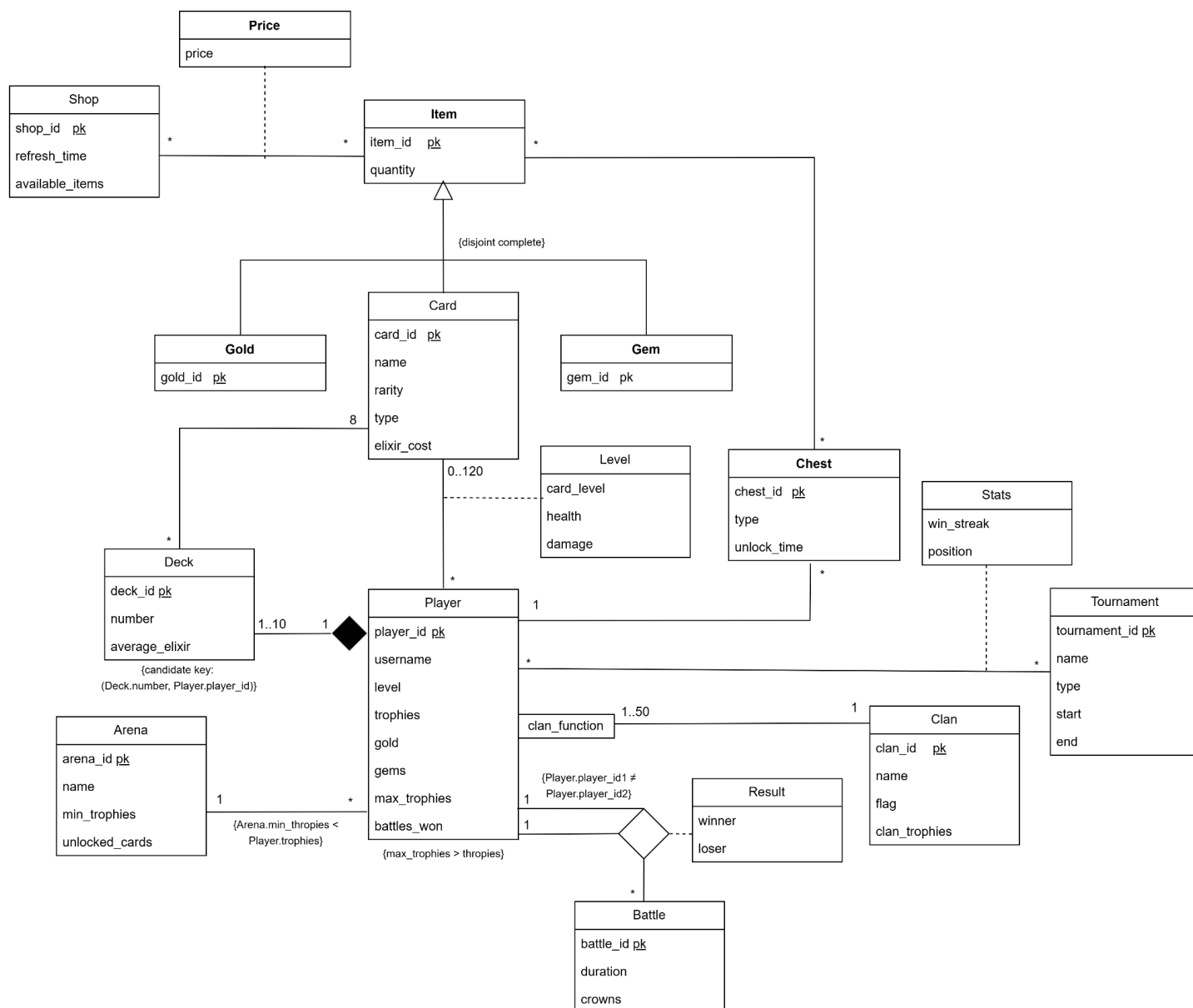
Rodrigo Henriques Roque : up202403623@up.pt

Índice

Índice	2
Modelo Conceptual Melhorado	3
Esquema Relacional	4
Proposta Inicial	4
Integração de IA Generativa	5
Proposta Final	7
Dependências Funcionais e Formais Normais	8
Proposta Inicial	8
Violações de Boyce-Codd	10
Violação da Terceira Forma Normal (3NF)	11
Integração de IA Generativa	12
Proposta Final	14
Criação da Base de Dados em SQLite	15
População da Base de Dados	17
Integração de IA Generativa	19
Conclusão	21
Links	22
Contribuição dos elementos do grupo	23

Modelo Conceptual Melhorado

Desenvolvido o nosso modelo conceitual da base de dados, a análise detalhada do modelo e o apoio do Professor Lázaro Gabriel Barros da Costa permitiram corrigir pequenas inconsistências que prepararam o prepararam definitivamente para armazenar e organizar de forma eficaz toda a informação relativa a jogadores, clãs, arenas, cartas, decks, batalhas, torneios, baús e loja, refletindo com rigor o funcionamento do jogo.



<https://drive.google.com/file/d/1AYCfco1IRgveuxELZW6CHa8BYj-BHfn3/view?usp=sharing>
(para mais qualidade é possível aceder a este link e abrir usando o draw.io)

Esquema Relacional

Proposta Inicial

Propusemos um esquema relacional que traduz o modelo conceptual do sistema para uma estrutura que garante a integridade e a consistência dos dados. As entidades principais **Shop**, **Item**, **Card**, **Gold**, **Gem**, **Player**, **Deck**, **Arena**, **Chest**, **Tournament**, **Clan** e **Battle** têm todas uma relação própria. As tabelas de associação, como **Price**, **Level**, **ItemChest**, **CardDeck**, **DeckPlayer**, **PlayerChest**, **Stats**, **PlayerArena** e **PlayerClan**, permitem modelar todas as relações, ligando entidades e armazenando informações adicionais, como preços, níveis de cartas ou funções de clãs.

O esquema relacional representa a estrutura e todas as relações e dependências definidas no modelo UML da seguinte forma, clara e eficiente:

Classes

- **Shop**(shop_id, refresh_time, available_items)
- **Item**(item_id, quantity)
- **Card**(item_id → Item, card_id, name, rarity, type, elixir_cost)
- **Gold**(item_id → Item, gold_id)
- **Gem**(item_id → Item, gem_id)
- **Player**(player_id, username, level, trophies, gold, gems, max_trophies, battles_won)
- **Deck**(deck_id, number, average_elixir)
- **Arena**(arena_id, name, min_trophies, unlocked_cards)
- **Chest**(chest_id, type, unlock_time)
- **Tournament**(tournament_id, name, type, start, end)
- **Clan**(clan_id, name, flag, clan trophies)
- **Battle**(battle_id, duration, crowns)

Relações

- **Price**(item_id → Item, shop_id → Shop, price)
- **Level**(player_id → Player, card_id → Card, card_level, health, damage)
- **ItemChest**(item_id → Item, chest_id → Chest)
- **CardDeck**(card_id → Card, deck_id → Deck)
- **DeckPlayer**(deck_id → Deck, player_id → Player)
- **PlayerChest**(chest_id → Chest, player_id → Player)
- **Stats**(player_id → Player, tournament_id → Tournament, win_streak, position)
- **PlayerArena**(player_id → Player, arena_id → Arena)
- **PlayerClan**(player_id → Player, clan_id → Clan, clan_function) {player_id, clan_function} UK
- **Result**(player_id1 → Player, player_id2 → Player, battle_id → Battle, loser, winner)

Integração de IA Generativa

Apresentamos o nosso esquema relacional a outra Inteligência Artificial Generativa, cedendo-lhe o nosso modelo UML final, o nosso esquema relacional e uma cópia do [04.UML Relations.pdf](https://docs.google.com/document/d/1Ybyl73Nuo0Ub13Ud77nDzZ-Aa_NKEImOJwdEmvzhNAI/edit?usp=sharing) como referência. Desta vez recorremos ao **Gemini** da Google, por ter acesso a informações mais recentes, manter uma boa capacidade de tratar tarefas que envolvem muitos dados ou múltiplas etapas e por analisar e estruturar os conteúdos de uma forma relativamente mais única do que outros modelos.

Conversa:

https://docs.google.com/document/d/1Ybyl73Nuo0Ub13Ud77nDzZ-Aa_NKEImOJwdEmvzhNAI/edit?usp=sharing

Usando o modo **Thinking** do modelo **3 Pro** inserimos a seguinte prompt:

“Assume que és um professor universitário. Utiliza linguagem rigorosa e um pensamento profundo. Baseia a tua resposta nas boas práticas da Linguagem Relacional aplicada a bases de dados e da Conversão de UML para Modelos Relacional, disponíveis no ficheiro 04.UMLToRelations anexado. Considera este modelo relacional inicial de uma base de dados do jogo Clash Royale em anexo. Considera ainda o modelo UML em anexo. Faz uma análise da correção e qualidade do modelo relacional, listando todos os pontos positivos e negativos.”

Recebemos um conjunto de comentários **com que concordamos**:

- *“O modelo introduz uma nova chave primária (card_id, gold_id, gem_id) que é desnecessária e semanticamente incorreta. A entidade Card é um Item; a sua chave primária deve ser a mesma, item_id. etc. A relação Level referencia card_id, que não deveria existir. Deveria referenciar item_id, a chave primária correta da entidade Card.”*

As subclasses **Card**, **Gem** e **Gold** herdam a identidade da classe **Item**, uma vez que apesar de lhe acrescentarem atributos, representando tipos específicos desta entidade. Como tal, **não necessitam** de uma chave primária própria, uma vez que são identificadas unicamente por **item_id**. Assim, estas chaves devem ser **removidas** e as relações, como **Level**, devem referenciar **item_id**, garantindo consistência e integridade referencial.

- *“Shop.available_items: Este atributo é redundante. A relação Price já modela quais itens estão disponíveis em qual loja (e a que preço). Manter este atributo na tabela Shop cria redundância de dados e potenciais problemas de consistência.”*

De facto, a relação **Price** já representa a associação entre lojas e os itens disponíveis, incluindo o respetivo preço. Manter este atributo na entidade **Shop** duplicaria informação e poderia originar inconsistências entre os dados. Assim, a sua remoção elimina redundância e assegura uma modelação mais coerente e normalizada.

- “Deck pertence a apenas um Jogador. Vocês representaram isto duas vezes: uma como atributo em Deck e outra como tabela DeckPlayer. Mantenham apenas o atributo `player_id` na tabela Deck e eliminem a relação DeckPlayer.”

É redundante ter em Deck uma chave estrangeira para Player (`player_id`) que associe uma instância de um baralho a um jogador e ter simultaneamente uma tabela para representar esta ligação. Deste modo, devemos excluir DeckPlayer.

E outros com que **não concordamos**:

- “A metodologia padrão, mais comum e mais eficiente para mapear uma associação 1:N é **adicionar uma chave estrangeira (FK) na tabela correspondente ao lado ‘N’**, que referencia a chave primária da tabela do lado ‘1’. O modelo proposto ignora esta regra e, em vez disso, cria relações de junção separadas, como se fossem associações M:N.

Apesar de termos em conta que é uma abordagem válida e, possivelmente, mais eficiente, optámos por **não seguir** esta metodologia. O modelo normalizado fornecido nos **conteúdos** e nas **aulas** propõe a criação de uma classe ou tabela **separada** para estas associações, mesmo quando se trata de relações 1:N. Esta abordagem, embora diferente da “prática padrão”, facilita a extensão futura do modelo e garante consistência no tratamento de associações complexas.

- “O atributo `clan_function`, deveria ser simplesmente mais um atributo na relação Player, juntamente com a `clan_id`.”

Por outras palavras, **clan_function** deveria ser um atributo da relação **Player**, assim como `clan_function`. Porém, este atributo existe apenas quando há uma ligação entre Player e Clan, funcionando como um **qualificador** que permite distinguir os jogadores dentro de um Clan associando-lhes uma função específica. Além disso, como referido anteriormente, optamos por criar relações para descrever as associações de um-para-muitos de modo a representá-las de forma normalizada.

Proposta Final

Após análise dos comentários e sugestões do Gemini, aperfeiçoamos o esquema relacional final do sistema, mantendo a coerência com o modelo UML. As entidades principais e as tabelas de associação refletem todas as relações e dependências relevantes, garantindo integridade, consistência e flexibilidade para a implementação da base de dados em SQL.

Classes

- **Shop**(shop_id, refresh_time)
- **Item**(item_id, quantity)
- **Card**(item_id → Item, name, rarity, type, elixir_cost)
- **Gold**(item_id → Item)
- **Gem**(item_id → Item)
- **Player**(player_id, username, level, trophies, gold, gems, max_trophies, battles_won)
- **Deck**(deck_id, number, average_elixir, player_id → Player)
- **Arena**(arena_id, name, min_trophies, unlocked_cards)
- **Chest**(chest_id, type, unlock_time)
- **Tournament**(tournament_id, name, type, start, end)
- **Clan**(clan_id, name, flag, clan_trophies)
- **Battle**(battle_id, duration, crowns)

Relações

- **Price**(item_id → Item, shop_id → Shop, price)
- **Level**(player_id → Player, item_id → Card, card_level, health, damage)
- **ItemChest**(item_id → Item, chest_id → Chest)
- **CardDeck**(item_id → Card, deck_id → Deck)
- **PlayerChest**(chest_id → Chest, player_id → Player)
- **Stats**(player_id → Player, tournament_id → Tournament, win_streak, position)
- **PlayerArena**(player_id → Player, arena_id → Arena)
- **PlayerClan**(player_id → Player, clan_id → Clan, clan_function) {player_id, clan_function} *UK*
- **Result**(player_id1 → Player, player_id2 → Player, battle_id → Battle, loser, winner)

Dependências Funcionais e Formais Normais

Estas dependências funcionais resultam diretamente das relações entre atributos observadas no esquema relacional final. A partir desta análise, pudemos validar a normalização, garantir a integridade dos dados e otimizar a estrutura da base de dados.

Proposta Inicial

Shop(shop_id, refresh_time)

- $\text{shop_id} \rightarrow \text{refresh_time}$

Item(item_id, quantity)

- $\text{item_id} \rightarrow \text{quantity}$

Card(item_id $\rightarrow$ Item, name, rarity, type, elixir_cost)

- $\text{item_id} \rightarrow \text{name, rarity, type, elixir_cost}$;

Gold(item_id $\rightarrow$ Item)

- não existem dependências funcionais não triviais

Gem(item_id $\rightarrow$ Item)

- não existem dependências funcionais não triviais

Player(player_id, username, level, trophies, gold, gems, max_trophies, battles_won)

- $\text{player_id} \rightarrow \text{username, level, trophies, gold, gems, max_trophies, battles_won}$

Deck(deck_id, number, average_elixir, player_id $\rightarrow$ Player)

- $\text{deck_id} \rightarrow \text{number, average_elixir, player_id}$

Arena(arena_id, name, min_trophies, unlocked_cards)

- $\text{arena_id} \rightarrow \text{name, min_trophies, unlocked_cards}$
- $\text{name} \rightarrow \text{arena_id, min_trophies, unlocked_cards}$
- $\text{min_trophies} \rightarrow \text{arena_id, name, unlocked_cards}$

Chest(chest_id, type, unlock_time)

- $\text{chest_id} \rightarrow \text{type, unlock_time}$
- $\text{type} \rightarrow \text{unlock_time}$

Tournament(tournament_id, name, type, start, end)

- $\text{tournament_id} \rightarrow \text{name, type, start, end}$

Clan(clan_id, name, flag, clan_trophies)

- $\text{clan_id} \rightarrow \text{name, flag, clan_trophies}$

Battle(battle_id, duration, crowns)

- $\text{battle_id} \rightarrow \text{duration, crowns}$

Price(item_id $\rightarrow$ Item, shop_id $\rightarrow$ Shop, price)

- $\text{item_id} \rightarrow \text{shop_id, price}$

Level(player_id $\rightarrow$ Player, item_id $\rightarrow$ Card, card_level, health, damage)

- $(\text{player_id, item_id}) \rightarrow \text{card_level, health, damage}$

ItemChest(item_id $\rightarrow$ Item, chest_id $\rightarrow$ Chest)

- não existem dependências funcionais não triviais

CardDeck(item_id $\rightarrow$ Card, deck_id $\rightarrow$ Deck)

- não existem dependências funcionais não triviais

PlayerChest(chest_id $\rightarrow$ Chest, player_id $\rightarrow$ Player)

- $\text{chest_id} \rightarrow \text{player_id}$

Stats(player_id $\rightarrow$ Player, tournament_id $\rightarrow$ Tournament, win_streak, position)

- $(\text{player_id, tournament_id}) \rightarrow \text{win_streak, position}$

PlayerArena(player_id $\rightarrow$ Player, arena_id $\rightarrow$ Arena)

- $\text{player_id} \rightarrow \text{arena_id}$

PlayerClan(player_id $\rightarrow$ Player, clan_id $\rightarrow$ Clan, clan_function) {player_id, clan_function} *UK*

- $\text{player_id} \rightarrow \text{clan_id, clan_function}$

Result(player_id1 $\rightarrow$ Player, player_id2 $\rightarrow$ Player, battle_id $\rightarrow$ Battle, loser, winner)

- $\text{battle_id} \rightarrow \text{player_id1, player_id2, loser, winner}$

Violações de Boyce-Codd

Uma relação R está na forma normal Boyce-Codd se sempre que existe uma dependência funcional não trivial, isto é uma dependência do tipo $A \rightarrow A$, o lado esquerdo da dependência é uma super-chave para a relação R.

Analisando todas as nossas relações, a única que apresenta uma dependência funcional na qual o lado esquerdo não é uma superchave é a relação **Chest**. Nesta relação, `type` determina o `unlock_time`, visto que baús do mesmo tipo têm o mesmo tempo de desbloqueio, mas não determina o `chest_id`, isto porque o este ID é único e existem vários baús do mesmo tipo com IDs diferentes. Assim sendo, o `type` não pode ser uma super-chave o que resulta numa violação da forma normal.

Eliminando a violação, inicialmente temos:

Chest(chest_id, type, unlock_time)

- $\text{chest_id} \rightarrow \text{type}, \text{unlock_time}$
- $\text{type} \rightarrow \text{unlock_time}$

$\text{type}^+ = \{\text{type}, \text{unlock_time}\}$

Decompomos a relação **Chest** em:

- **ChestType**(type, unlock_time)
- **ChestInstance**(chest_id, type)

As dependência funcionais nestas passam a ser:

ChestType(type, unlock_time):

- $\text{type} \rightarrow \text{unlock_time}$

ChestInstance(chest_id, type)

- $\text{chest_id} \rightarrow \text{type}$;

Ora estas duas relações já se encontram na forma normal Boyce-Codd, por isso podemos parar a decomposição por aqui.

Violação da Terceira Forma Normal (3NF)

Uma relação R está na sua 3ª forma normal se sempre que existe uma dependência funcional não trivial, isto é uma dependência do tipo $A \rightarrow B$, ou o lado esquerdo da dependência é uma superchave para a relação R, ou os membros do lado direito que não estão do lado esquerdo são todos primos, ou seja são atributos de uma chave. Tal implica que um atributo não-chave não pode depender de outro atributo não-chave.

Podemos interpretar esta forma normal como uma forma mais liberal da forma normal Boyce-Codd, portanto se vimos anteriormente que apenas a relação **Chest** falhava, então ao analisar as violações da 3NF apenas nos temos de preocupar com esta. Como temos a dependência $\text{type} \rightarrow \text{unlock_time}$, e o unlock_time não é um atributo de uma chave, então concluímos que a relação **Chest** inicial também não se encontra na 3ª forma normal.

Eliminando a violação, inicialmente temos:

Chest(chest_id, type, unlock_time)

- $\text{chest_id} \rightarrow \text{type}, \text{unlock_time}$
- $\text{type} \rightarrow \text{unlock_time}$

Decompomos a relação **Chest** em:

- **ChestInstance**(chest_id, type, unlock_time)
- **ChestType**(type, unlock_time)

As dependências funcionais nestas passam a ser:

ChestInstance(chest_id, type, unlock_time):

- $\text{chest_id} \rightarrow \text{type}, \text{unlock_time}$

ChestType(type, unlock_time):

- $\text{type} \rightarrow \text{unlock_time};$

Ora a relação **ChestInstance** é uma super-chave da relação **Chest** logo podemos acabar por aqui.

Integração de IA Generativa

Criamos uma nova conversa no **Gemini** da Google e utilizando o modelo **3 Pro**, que nos é facultado pela Universidade, e o seu modo **Thinking**, para reforçar e melhorar o nosso modelo relacional. Pedimos-lhe que analisasse o nosso modelo relacional e decomposição inicial de modo a garantir a criação apenas das dependências funcionais estritamente necessárias sem que haja perdas por decomposição.

Conversa:

<https://docs.google.com/document/d/1VavVXPPqbcX8ix8Aeqt5Q2VIBEBbbPuDkh7z-ul3Ej0/edit?usp=sharing>

Introduzimos no Gemini A seguinte prompt:

“Assume que és um professor universitário. Utiliza linguagem rigorosa e um pensamento profundo. Baseia a tua resposta nas boas práticas da Teoria de Design Relacional aplicada a bases de dados. Considera este modelo relacional de uma base de dados do jogo Clash Royale. Analisa o modelo de Dependências Funcionais em anexo estritamente com base no modelo relacional fornecido também em anexo e faz uma análise da sua correção, qualidade e adequação ao modelo relacional, listando todos os pontos positivos e negativos. Repara ainda em possíveis violações da Forma Normal de Boyce-Codd e na 3ª Forma Normal, além das que nós analisamos e corrigimos.”

Analisando criticamente a sua resposta, **concordamos** com os seguintes pontos:

- *“Os atributos health e damage dependem exclusivamente da carta e do seu nível, e não de quem é o jogador. Temos aqui uma Dependência Transitiva:*
 - 1. (player_id, item_id) → card_level (O jogador tem a carta a um certo nível).*
 - 2. (item_id, card_level) → health, damage (Uma carta naquele nível tem sempre aqueles atributos).*

Solução Recomendada:

Decompor em:

CardStats(item_id, level, health, damage)

PlayerCardLevel(player_id, item_id, current_level)”

A nossa proposta inicial viola a 3ª Forma Normal, sendo que existem atributos não-chave (*health, damage*) que dependem de outro atributo não-chave (*card_level*) (parcialmente, em conjunto com a identidade da carta). Isto gera redundância: sempre que dois jogadores têm, por exemplo, um "Gigante" de nível 9, repetem-se os valores de vida e dano na base de dados.

- “Se *item_id* determina o *shop_id*, isso implica funcionalmente que um item só pode ser vendido numa única loja. Se a vossa intenção era modelar que um item aparece na loja diária, a chave primária deveria ser composta (*item_id*, *shop_id*) ou apenas o *shop_id* se a loja tiver slots fixos.”

Um item pode estar disponível em várias lojas a preços diferentes e cada loja vai sendo atualizada regularmente com novas ofertas. Assim, o preço deve depender simultaneamente do item em questão e da loja em que surge a oferta, sendo necessário que a chave primária de *Price* tenha os atributos *item_id* e *shop_id*.

E discordamos com:

- “O atributo *unlocked_cards* parece ser multivalorado (uma lista de cartas). Se for uma string separada por vírgulas, viola a 1ª Forma Normal (atomicidade). Se for apenas o número de cartas, tudo bem. Se a intenção é saber quais cartas desbloqueiam, é necessária uma tabela associativa *ArenaCards*(*arena_id*, *card_id*), caso contrário, a dependência funcional não captura a relação M:N entre Arenas e Cartas.”

Precisamente, *unlocked_cards* representa apenas o número de cartas a serem desbloqueadas ao alcançar uma nova arena, não violando a 1FN.

Proposta Final

Após a decomposição e normalização das relações que concebemos inicialmente e da aplicação das sugestões da Inteligência Artificial Generativa com que concordamos, construímos o seguinte esquema:

Classes

- **Shop**(shop_id, refresh_time)
- **Item**(item_id, quantity)
- **Card**(item_id → Item, name, rarity, type, elixir_cost)
- **Gold**(item_id → Item)
- **Gem**(item_id → Item)
- **Player**(player_id, username, level, trophies, gold, gems, max_trophies, battles_won)
- **Deck**(deck_id, number, average_elixir, player_id → Player)
- **Arena**(arena_id, name, min_trophies, unlocked_cards)
- **ChestType**(type, unlock_time)
- **ChestInstance**(chest_id, type)
- **Tournament**(tournament_id, name, type, start, end)
- **Clan**(clan_id, name, flag, clan_trophies)
- **Battle**(battle_id, duration, crowns)

Relações

- **Price**(item_id → Item, shop_id → Shop, price)
- **CardStats**(item_id → Card, level, health, damage)
- **PlayerCardLevel**(player_id, item_id, current_level)
- **ItemChest**(item_id → Item, chest_id → Chest)
- **CardDeck**(item_id → Card, deck_id → Deck)
- **PlayerChest**(chest_id → Chest, player_id → Player)
- **Stats**(player_id → Player, tournament_id → Tournament, win_streak, position)
- **PlayerArena**(player_id → Player, arena_id → Arena)
- **PlayerClan**(player_id → Player, clan_id → Clan, clan_function) {player_id, clan_function} UK
- **Result**(player_id1 → Player, player_id2 → Player, battle_id → Battle, loser, winner)

Criação da Base de Dados em SQLite

Para desenvolver e otimizar a base de dados, recorreremos novamente ao modelo **3 Pro do Gemini**. Instruímos-lo de forma a focar-se nas boas práticas da linguagem SQLite e fornecemos-lhe o conteúdo do nosso ficheiro [Create1.sql](#) com a criação das tabelas

Conversa: <https://gemini.google.com/share/ea17f3bb4d9b>

A nossa prompt inicial foi:

“Assume que és um professor universitário. Utiliza linguagem rigorosa e um pensamento profundo. Baseia a tua resposta nas boas práticas da linguagem sqlite aplicada a bases de dados. Considera este código de uma base de dados do jogo Clash Royale. Analisa o modelo e faz uma análise da sua correção, qualidade e adequação ao modelo relacional, listando todos os pontos positivos e negativos. Gera um código novo, com comentários nos aspetos que corrigiste. Não adiciones triggers nem mudes a estrutura das tabelas e atributos. Aqui esta o codigo: (...)”

A análise da inteligência artificial devolveu um comentário positivo:

“Integridade de Domínio (CHECK): A tua utilização extensiva de cláusulas CHECK é louvável. Estás a garantir que o elixir não passa de 10, que as quantidades são positivas e que as datas têm lógica cronológica. Isso é fundamental para manter a sanidade dos dados. Normalização e Herança: Implementaste corretamente uma estratégia de Class Table Inheritance (Herança por Tabela de Classes) para a entidade Item e as suas subclasses (Card, Gold, Gem). É uma abordagem relacional robusta. Integridade Referencial: A ativação de PRAGMA foreign_keys = ON; é o primeiro passo para uma base de dados SQLite séria. Sem isso, as tuas Foreign Keys seriam meramente decorativas.”

Mas também devolveu algumas sugestões e gerou um código corrigido do qual nós aceitamos algumas das correções, como:

- *“Substituí todos os VARCHAR, INT (quando não PK) e DATETIME pelos tipos nativos do SQLite: TEXT e INTEGER. Porquê: O SQLite utiliza um sistema de Type Affinity. Diferente do PostgreSQL ou MySQL, o tipo VARCHAR(255) não existe fisicamente; o SQLite converte-o para TEXT. O tipo DATETIME também não existe; datas são guardadas como TEXT (String ISO8601), REAL ou INTEGER. Usar os nomes nativos evita a ilusão de que a base de dados está a validar limites de caracteres ou formatos de data.”*
- *“Altereí a definição das chaves primárias de INT PRIMARY KEY para INTEGER PRIMARY KEY (escrito por extenso). Porquê: Esta é a mudança mais crítica para a performance em SQLite. INT PRIMARY KEY: Cria um índice B-Tree para a chave primária E mantém uma tabela separada com um ROWID oculto. O motor tem de fazer duas pesquisas para chegar aos dados. INTEGER PRIMARY KEY: Torna a coluna um alias para o ROWID interno.”*

Ambas as sugestões de otimização visam melhorar a eficiência da base de dados e adequá-la o mais possível ao SQLite. Assim, aumentamos a velocidade de acesso aos dados e usamos um formato de texto mais flexível sem um comprimento máximo fixo, além de evitar possíveis erros de interpretação por parte da linguagem. No entanto, optamos por não alterar o tipo de dados DATETIME para TEXT ou REAL para manter a clareza e legibilidade, apesar do SQLite não possuir um tipo de data nativo.

- *“Adicionei ON DELETE CASCADE nas tabelas filhas da herança (Card, Gold, Gem). Porquê: Estamos a usar uma estratégia de “Herança por Tabelas”. Se apagares um registo na tabela pai (Item), o registo correspondente na tabela filha (Card) ficaria “órfão” sem esta instrução. O CASCADE garante que a limpeza do pai elimina automaticamente os filhos, mantendo a base de dados higienizada.”*

A cláusula ON DELETE CASCADE é essencial na estratégia de herança para garantir a integridade dos dados. Desta forma asseguramos que ao remover uma linha na tabela pai *Item* eliminamos os registos correspondentes nas tabelas filhas *Card*, *Gem* e *Gold*, evitando a existência de registos sem significado nestas tabelas.

E refutamos outras, como:

- *“Adicionei Foreign Keys (FK) para as colunas winner_id e loser_id e adicionei cláusulas CHECK lógicas. Porquê: FKs: Antes, podias inserir um ID de vencedor que não existia na tabela Player (um ‘ID fantasma’). Agora, a base de dados rejeita IDs inválidos. CHECKs: Adicionei a regra CHECK (winner_id = player_id1 OR winner_id = player_id2). Isto impede inconsistências lógicas, como o ‘Jogador A’ enfrentar o ‘Jogador B’, mas o vencedor registado ser o ‘Jogador C’ (que nem estava na batalha).”*

Embora tecnicamente correto, adicionar FK e CHECKs em winner_id cria redundância e rigidez excessiva. Entendemos a intenção de proteger os dados, mas essa abordagem complica demasiado a criação das tabelas apenas para resolver um problema quase nulo. Isto pode causar problemas caso precisemos de ajustar os dados manualmente ou permitir coisas simples como um empate.

- *“Removi a restrição CONSTRAINT uk_player_role UNIQUE (player_id, clan_role). Porquê: A tabela já tinha player_id como PRIMARY KEY. Por definição, uma chave primária é única. Se o player_id 10 só pode aparecer uma vez na tabela, a combinação (10, 'Leader') também só aparecerá uma vez. A restrição UNIQUE adicional era ruído computacional desnecessário.”*

O objetivo da restrição não é garantir a unicidade do player_id para que cada jogador pertença apenas a um clã. Embora essa condição exista, ela já é assegurada pelo facto de o player_id ser uma Chave Primária, como a IA referiu. A restrição serve, sim, para garantir a exclusividade de uma função dentro do clã, assegurando que apenas um jogador possa deter um cargo específico (como, por exemplo, 'Líder') dentro desse grupo.

População da Base de Dados

Depois de prepararmos uma população inicial para introduzir na base de dados recorremos à inteligência artificial para analisar a qualidade do nosso sistema e nos ajudar a inserir alguns dados mais variados. Para o efeito utilizamos mais uma vez o modo **Thinking** do **Gemini 3 Pro**, mantendo uma certa coerência no tratamento dos dados.

Conversa: <https://gemini.google.com/share/12474fca00b6>

Fornecemos ao modelo o ficheiro [Populate1.sql](#), com o código em sql para introduzir os nossos dados em todas as tabelas previamente criadas, e a promot seguinte:

“Assume que és um professor universitário. Utiliza linguagem rigorosa e um pensamento profundo. Baseia a tua resposta nas boas práticas da linguagem sqlite aplicada a bases de dados. Considera este código da população de uma base de dados do jogo Clash Royale. Analisa o modelo e faz uma análise da sua correção e qualidade, listando todos os pontos positivos e negativos. Gera um código novo, com comentários nos aspetos que corrigiste. Não adiciones triggers nem mudes a estrutura das tabelas e atributos.”

A partir do nosso código foi-nos devolvida uma lista de aspetos positivos, como:

“1. Integridade Referencial Explícita: O uso de PRAGMA foreign_keys = ON; logo no início é uma prática excelente e obrigatória em SQLite para garantir que o SGBD valide as relações entre tabelas. 2. Tratamento de Strings: A utilização de escaping correto para apóstrofes (ex: 'P.E.K.K.A's Playhouse') demonstra atenção à sintaxe SQL padrão. 3. Cobertura do Domínio: O dataset cobre todas as entidades principais (Cartas, Arenas, Jogadores, Batalhas), permitindo testar queries complexas.”

Além disso, recebemos algumas sugestões de melhoria, das quais adotamos:

- *“Na tabela Card, usas 'ground', 'aerial', 'building'. Na tabela Tournament, usas 'Elimination', 'Round Robin' (Capitalized). Na tabela PlayerClan, usas 'Leader', 'Member'.”*

É uma questão pertinente de formatação, faz sentido seguir um padrão uniforme para nomear os valores dos atributos de várias tabelas, facilitando a leitura e interpretação do código. Formatamos algumas entradas de acordo com as convenções do SQL, *mas mantivemos outras para respeitar certas restrições das tabelas da base de dados.*

- *“Tens batalhas com duração 300 (5 minutos). No Clash Royale, o tempo base é 3 minutos. 5 minutos implica Overtime (Prolongamento) completo. Tens as batalhas 21 e 22 com duração 300 e crowns_total = 0. Isto significa que jogaram 5 minutos e ninguém derrubou nenhuma torre. É um empate (Draw). No entanto, tens a Batalha 4 com 300s e 0 coroas. Estatisticamente, tantos empates de 5 minutos são raros. Para dados de teste mais realistas, varia as durações. A maioria das batalhas termina antes dos 300s (morte súbita).”*

A crítica é válida e aponta uma anomalia relativamente importante nos dados de teste. Realmente, um empate é um evento muito raro uma vez que a maioria dos jogos não atinge esse limite. Neste contexto, a solução mais adequada é inserir mais batalhas e mais diversificadas para representar o ambiente natural do jogo.

- *“Atomicidade (Ausência de Transações): O script executa centenas de INSERT isolados. Em SQLite, cada instrução é uma transação automática, o que é extremamente lento (I/O intensivo) e perigoso. Se o script falhar a meio, a base de dados fica num estado inconsistente. Correção: Envolver tudo num bloco BEGIN TRANSACTION ... COMMIT.”*

Esta sugestão aborda a performance e a integridade da inserção de dados. Uma vez que a cada INSERT o sql precisa de abrir, escrever e fechar a base de dados, envolver os INSERTs numa transação única reduz drasticamente as operações de escrita no disco. Além disso, assim tornamos a inserção uma operação atômica, ou seja em caso de falha a meio, a base de dados reverte ao estado inicial, evitando dados parciais ou inconsistentes.

E outras que rejeitamos:

- *“Mistura de Domínios em Item: A tabela Item parece atuar como uma super-classe para Card e Resource. A lógica de criar itens específicos para quantidades específicas de recursos (ex: Item 112 é "100.000 Ouro") é uma modelação idiossincrática (normalmente a quantidade estaria na tabela de relação ItemChest).”*

A tabela *Item* funciona como um catálogo que regista uma instância de produto e a sua quantidade como um pacote. Deste modo, cada oferta de uma loja e cada recompensa de um baú é uma unidade atômica, por exemplo 100 de ouro, 1000 de ouro ou 50000 de ouro, e não apenas um recurso numa quantidade arbitrária. Assim, não interessa definir a quantidade de um recurso no seu registo em *Price* ou *ItemChest*, mas sim definir o pacote quantidade e recurso em *Item* e indicar a sua disponibilidade numa loja ou num baú.

- *“Inconsistência Semântica (Tabela Deck): Inseriste o valor 0.0 para o average_elixir do Deck 1, enquanto o comentário calculava manualmente um valor (incorreto, diga-se de passagem). Dados derivados armazenados devem refletir a realidade dos dados brutos, sob pena de invalidar a lógica de negócio.”*

A sugestão não se aplica porque os Decks são imutáveis, ou seja, uma alteração implica sempre gerar um novo registo, dispensando atualizações. Manter um baralho dinâmico que calculasse automaticamente a média de elixir com base nas suas cartas exigiria *triggers*, cujo uso não é um objetivo do projeto. Assim, o cálculo é garantido exclusivamente no momento da inserção.

Integração de IA Generativa

Os modelos **ChatGPT-5** e **Gemini 3 Pro** no **modo Thinking** funcionaram como parceiro de revisão interativa ao longo de todo o projeto. A sua utilização permitiu esclarecer várias dúvidas e corrigir pequenas nuances de uma forma muito mais célere e simples, embora seja sempre necessário um controlo rigoroso do seu retorno para filtrar alucinações e soluções mal elaboradas ou excessivamente complexas. Assim, podemos separar as questões mais delicadas e complexas e estudá-las com maior foco com base no material teórico e com o auxílio do professor.

Na fase inicial de modelação o **ChatGPT-5** facilitou a **revisão crítica** do nosso diagrama UML. A ferramenta destacou a necessidade de refinar entidades como os níveis das cartas e a identificação dos baralhos, detetando ambiguidades estruturais. Por outro lado, tivemos de filtrar sugestões resultantes de **erros de leitura da imagem** ou **desalinhadas** com as regras específicas do *Clash Royale* e do contexto do nosso projeto. A validação da IA permitiu-nos **corrigir inconsistências** e solidificar o modelo conceptual, garantindo uma base **lógica mais rigorosa** para o desenvolvimento subsequente da base de dados.

Já a **principal vantagem** do **Gemini** foi a **verificação da correção técnica e detecção de padrões**. O modelo destacou-se na identificação de:

- **Violações de Normalização:** A detecção da Dependência Transitiva nos atributos health e damage (que dependiam do nível da carta e não do jogador) foi crucial para atingir a 3ª Forma Normal.
- **Especificidades do SQLite:** A correção dos tipos de dados de INT para INTEGER e de VARCHAR e DATETIME para TEXT demonstraram um conhecimento profundo das particularidades do SQLite que poderiam passar despercebidas.
- **Análise Estatística de Dados:** A capacidade de inferir que "300 segundos com 0 coroas" é um empate e que a frequência elevada de empates é uma anomalia revelou uma "compreensão" surpreendente da lógica do domínio.

Este modelo foi especialmente útil na normalização e optimização do SQL. Nesta etapa, as regras como as Dependências Funcionais e Formas Normais são objetivas e matemáticas. Assim, a IA pôde detetar redundâncias subtis, como a gestão do atributo *available_items* de *Shop*, e sugerir optimizações de performance, como a indexação via INTEGER PRIMARY KEY, que elevaram a qualidade técnica do projeto. Aqui, a capacidade de processamento lógico da IA superou a revisão manual da documentação do SQL e funcionamento particular do SQLite.

No entanto, apresentou algumas **limitações principais** na **interpretação da intenção do design**. A IA tende a favorecer padrões e cumpri-los à risca em detrimento de escolhas arquiteturais deliberadas do nosso projeto:

1. **Rigidez Excessiva:** A sugestão de adicionar CHECK e constraints complexos para validar o vencedor da batalha, embora tecnicamente correta, introduz uma rigidez desnecessária que dificultaria a manutenção e testes manuais.
2. **Cegueira Contextual:** A dificuldade em aceitar a tabela Item como um catálogo unificado de produtos que nós elaboramos demonstrou que a ferramenta analisa a estrutura isoladamente, sem compreender totalmente a vantagem prática da unificação de inventário para o sistema de *loja* e *bundles*.

Assim, a IA foi menos eficaz na revisão do modelo relacional, uma vez que nesta fase, as decisões dependem da arquitetura que escolhemos seguir e da extensibilidade futura. A insistência da IA em transformar tabelas de relação 1:N em simples *Foreign Keys*, ou a sua crítica à modelação das funções do Clã, revelaram uma falta de sensibilidade para com as escolhas de *design* que visam a clareza e escalabilidade da base de dados. Nestes momentos, a adoção de boas práticas genéricas entrou em conflito com a realidade específica do nosso projeto, obrigando-nos a rejeitar muitas das suas sugestões para preservar a integridade e lógica das relações.

Conclusão

Concluindo, com este projeto conseguimos desenvolver e implementar uma base de dados relacional completa e normalizada para representar o ecossistema do jogo Clash Royale. Através da análise rigorosa das dependências funcionais e com algum apoio da Inteligência Artificial Generativa, foi possível corrigir violações das formas normais, refinar o esquema lógico e assegurar a integridade referencial na implementação em SQLite. O resultado final é um sistema robusto, capaz de armazenar e organizar de forma eficaz toda a informação relativa a jogadores, clãs, arenas, cartas, baralhos, batalhas, torneios e transações, refletindo com rigor o funcionamento e a complexidade do jogo.

Links

Os links foram integralmente atualizados em 23/11/2025

Imagem UP:

<https://www.up.pt/portal/media/images/feup.original.format-png.png>

Modelo UML Melhorado:

<https://drive.google.com/file/d/1AYCfco1IRgveuxELZW6CHa8BYj-BHfn3/view?usp=sharing>

04 Uml Relations presentation:

https://moodle2526.up.pt/pluginfile.php/78038/mod_resource/content/19/04.UMLToRelations.pdf

Conversa com o Gemini sobre o Modelo Relacional:

https://docs.google.com/document/d/1Ybyl73Nuo0Ub13Ud77nDzZ-Aa_NKEImOJwdEmvzhNAI/edit?usp=sharing

Conversa com o Gemini sobre as Dependências Funcionais e Forma Normal:

<https://docs.google.com/document/d/1VavVXPPqbcX8ix8Aeqt5Q2VIBEBbbPuDkh7z-ul3Ej0/edit?usp=sharing>

Create1.sql:

<https://docs.google.com/document/d/19RBIULy2LE47n6VMq9fHTXLxRvgJiQdWh6DEZOFFU8k/edit?usp=sharing>

Conversa com o Gemini sobre a Criação da Base de Dados em SQL:

<https://gemini.google.com/share/ea17f3bb4d9b>

Populate1.sql:

https://docs.google.com/document/d/1E37sfuMR8h9QPJgyAXexS_aMcFvDhhQEIpSiZbXQfmM/edit?usp=sharing

Conversa com o Gemini sobre a População da Base de Dados:

<https://gemini.google.com/share/12474fca00b6>

Create2.sql:

<https://docs.google.com/document/d/1ojOyJCmhGmgVJUa-eO8GBw36nvEVYiVTNKTffGuJCYU/edit?usp=sharing>

População2.sql:

https://docs.google.com/document/d/1eBgadY4aH65Y6lWysBtPPHM3T1DfvFYFW_qRGzJ2D0Y/edit?usp=sharing

Contribuição dos elementos do grupo

Os três elementos do grupo trabalharam de forma equilibrada e cooperativa, assegurando uma distribuição justa das tarefas ao longo do projeto. Cada membro contribuiu ativamente para as diferentes fases do trabalho, prestando apoio mútuo.

Francisco Lourenço - 33,(3)%

João Martins - 33,(3)%

Rodrigo Roque - 33,(3)%